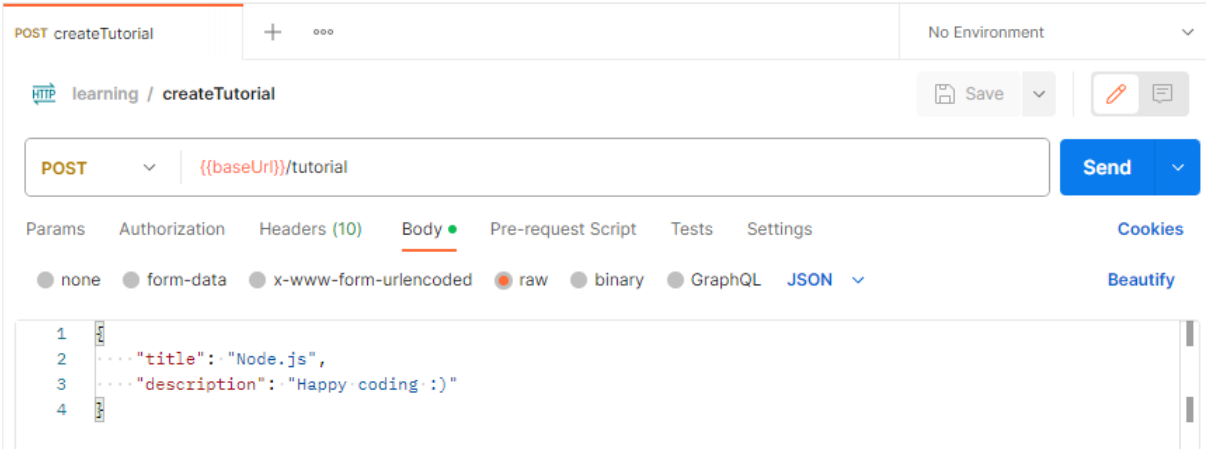
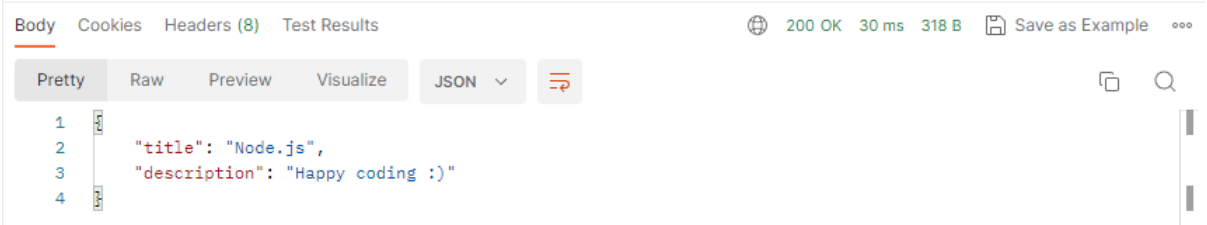
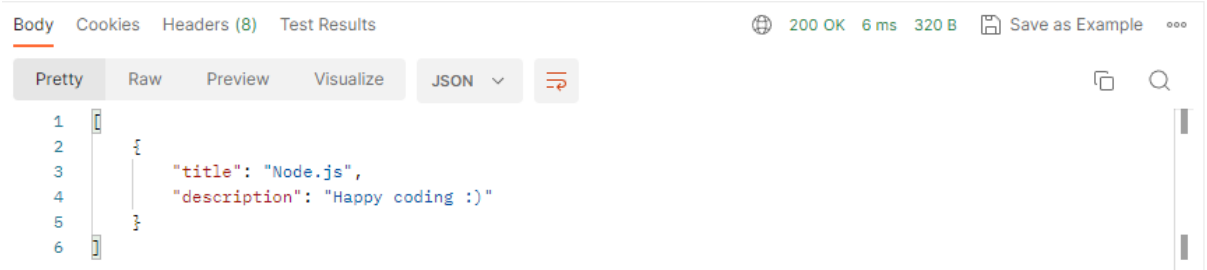
Lab 1: Node.js Web API

Learning Outcomes

- Create web server using Node.js and Express
- Implement web API Get and Post methods
- Use Postman to test Post and Get methods







Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

Software Required

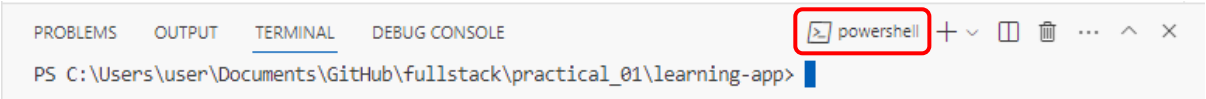
- Node.js (LTS version) https://nodejs.org/en/
- Visual Studio Code https://code.visualstudio.com/download
- Postman https://www.postman.com/downloads

Activity 1: Create web server using Node.js and Express

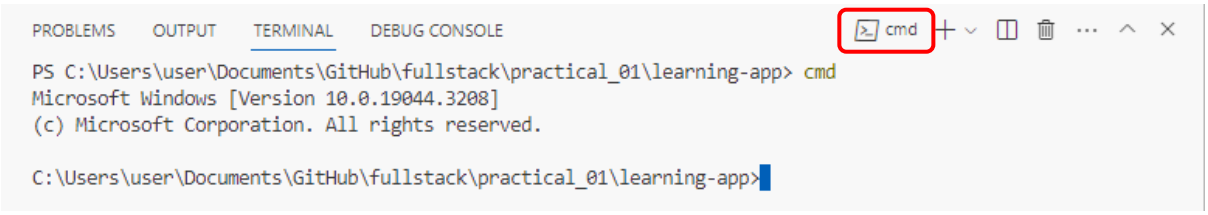
1. Create a new folder **learning-app**.

2. Open the folder in VS Code. Under learning-app, add a new folder **server**.

3. Open new terminal. If the default terminal profile is using powershell:

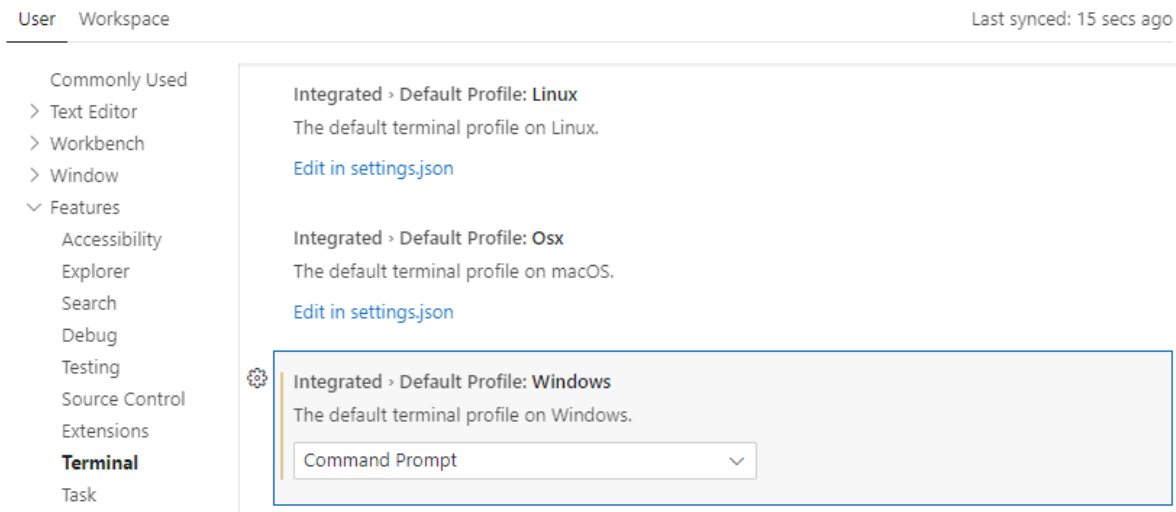


Run **cmd** to switch to cmd.



The advantage of using Command Prompt (cmd) is its simplicity and wide compatibility with various scripts and commands. Since Command Prompt has been around for a long time, it's well-established and widely understood.

4. You can also change the default terminal profile. Click menu File > Preferences > Settings, under User, look for Features > Terminal, edit Integrated > Default Profile to use Command Prompt:



Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

5. In terminal, run command to change directory to the server folder:
   cd server

6. Run command to initialize a Node.js project:
   npm init -y
   The file package.json is generated. It records important metadata about a project and also defines functional attributes of a project that npm uses to install dependencies, run scripts, and identify the entry point to our package.
   By default it indicates that **index.js** is the entry point.

7. Run command to install package express:
   npm install express
   or
   npm i express
   Express is used to create a web server in Node.js.
   Check content of the file package.json. Now the express version is added under dependencies.

8. Under server, add a new file **index.js**

```
const express = require('express');

const app = express();

// Simple Route
app.get("/", (req, res) => {
    res.send("Welcome to the learning space.");
});

let port = 3001;
app.listen(port, () => {
    console.log(`Sever running on http://localhost:${port}`);
});
```

This will start a web server with the url http://localhost:3001

9. In package.json, add the start script:

```
  "scripts": {
    "start": "node index.js"
  },
```

The script will be used by the **npm** command.

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

10. Run command to start the web server:
    npm start
    Open the url in the browser. You will see the welcome message.

11. In index.js, change the message. Refresh the web page, notice that the message is not updated.
    You need to press **Ctrl + C** to stop the server, and run command **npm start** to restart the server. Now you refresh the web page, the message is updated.

12. Run command to install package nodemon:
    npm i nodemon --save-dev
    Nodemon is a tool that helps develop Node.js based applications by automatically restarting the node application when file changes in the directory are detected.
    "--save-dev" is used in the command when the package is only used during development and the package version is added to package.json under devDependencies.

13. In package.json, update the start script to use nodemon:

```
  "scripts": {
    "start": "nodemon index.js"
  },
```

14. In index.js, change the message. Refresh the web page, notice that the message is updated now.

15. Press **Ctrl + C** to stop the server.

16. Run command to install package cors:
    npm i cors
    CORS stands for Cross-Origin Resource Sharing. It allows the client app to connect to the server API.

17. Run command to install package dotenv:
    npm i dotenv
    Dotenv is used to load environment variables from a .env file into process.env, storing configuration for the project.

18. Under server, add a new file **.env** for environment variables:

```
APP_PORT = 3001
CLIENT_URL="http://localhost:3000"
```

APP_PORT is used for the port number of the web API.
CLIENT_URL is used for the allowed origin for Cross-Origin Resource Sharing (CORS).

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

Note:
It is recommended to ignore the ".env" file in Git. This file often contains sensitive information like API keys and database credentials. By ignoring the ".env" file in Git, you help prevent the accidental exposure of these secrets in your version control system, which enhances your project's security. Instead, you can create a new file named ".env.example" with sample content to push to Git.

19. In index.js, add the require statement at the top of the file to read environment variable from the file .env:

```
require('dotenv').config();
```

Require the package to use process.env.[*variable*] to get the value.

20. In index.js, replace the hardcoded port number 3001 with the environment variable APP_PORT:

```
let port = process.env.APP_PORT;
```

21. In index.js, enable CORS:

```
const cors = require('cors');

const app = express();

// Enable CORS
app.use(cors({
    origin: process.env.CLIENT_URL
}));
```

Cross-Origin Resource Sharing (CORS) is a mechanism that allows web browsers to make requests to a different domain than the one the website is hosted on.

By default, web browsers enforce a security policy called the Same-Origin Policy, which restricts JavaScript code running in a web page from making requests to a different domain. This policy is in place to prevent malicious websites from accessing sensitive information from other websites without their permission.

If you don't enable CORS, you may see the error below in your browser console:

```
Access to XMLHttpRequest at 'https://localhost:7157/tutorial' localhost/:1
from origin 'http://localhost:3000' has been blocked by CORS policy: No
'Access-Control-Allow-Origin' header is present on the requested resource.
```

22. Run the server. Observe the URL of the web API is http://localhost:3001

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

Activity 2: Implement web API Get and Post methods

1. Under the folder server, add a new folder "**routes**"

2. Under the folder routes, add a new file **tutorial.js**

```
const express = require('express');
const router = express.Router();

let tutorialList = [];

router.post("/", (req, res) => {
    let data = req.body;
    tutorialList.push(data);
    res.json(data);
});

router.get("/", (req, res) => {
    res.json(tutorialList);
});

module.exports = router;
```

The variable **tutorialList** is an array to store the data in memory.
The **post** function is to add data into the list.
The **get** function is to list all the data.
The parameter **req** stands for request, **res** stands for response.
The function **res.json()** will set the response in json format.

3. In index.js, enable json format for request body:

```
const app = express();
app.use(express.json());
```

For the post function, we will read the request body in json format.

4. In index.js, add the new route.

```
// Simple Route
app.get("/", (req, res) => {
    res.send("Welcome to the learning space.");
});

// Routes
const tutorialRoute = require('./routes/tutorial');
app.use("/tutorial", tutorialRoute);
```

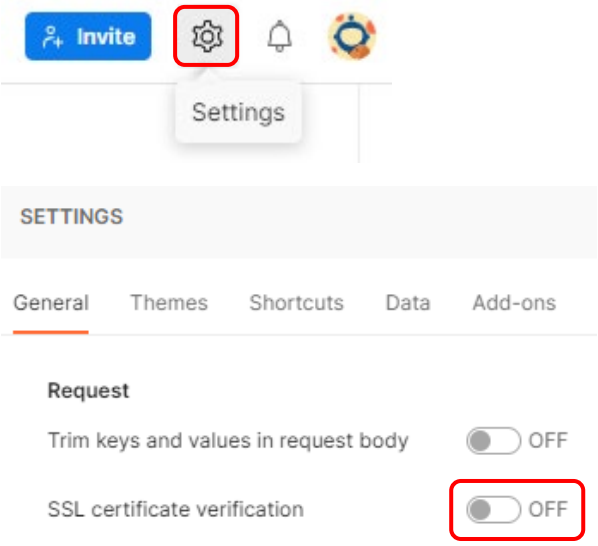
Require the tutorial route, set the route path starting with "/tutorial".

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

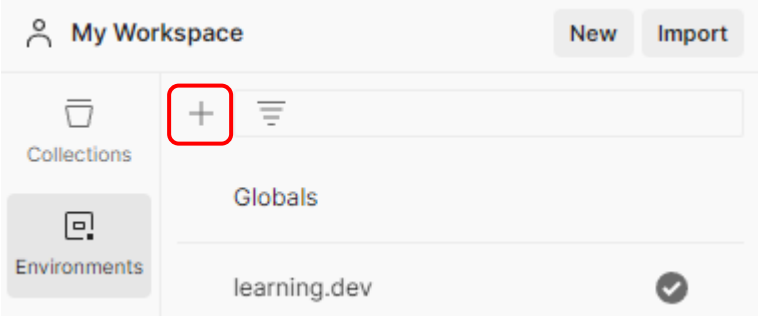
Activity 3: Test Get and Post methods

1. Open Postman, login to your account.

2. Open Settings, set **SSL certificate verification** to OFF.

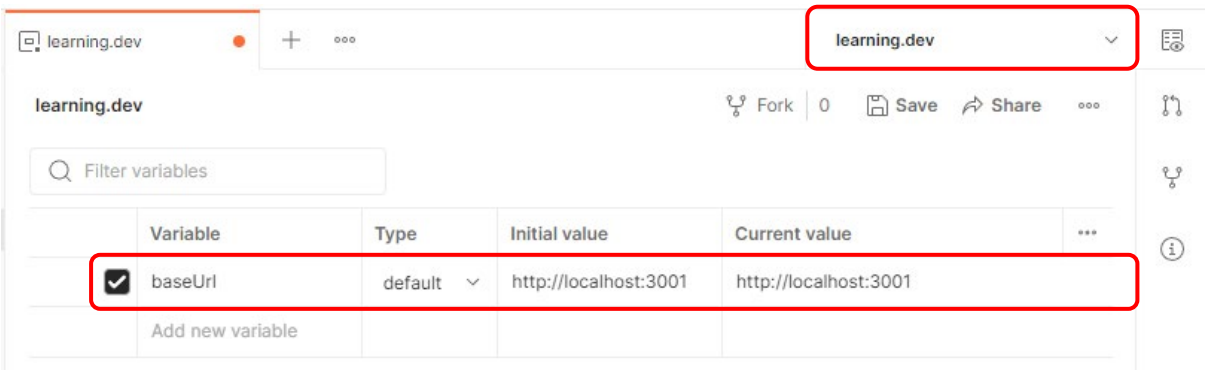


3. Under Environments, add a new environment **learning.dev**.



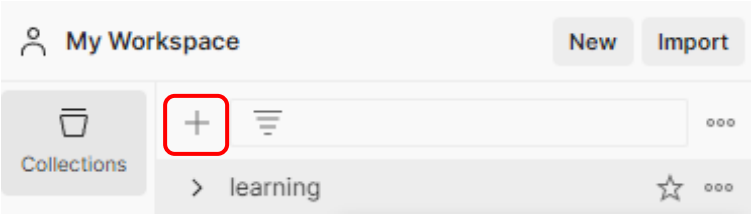
Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

4. Under the environment learning.dev, add a new variable **baseUrl** with value http://localhost:3001. Make sure that the environment is active.

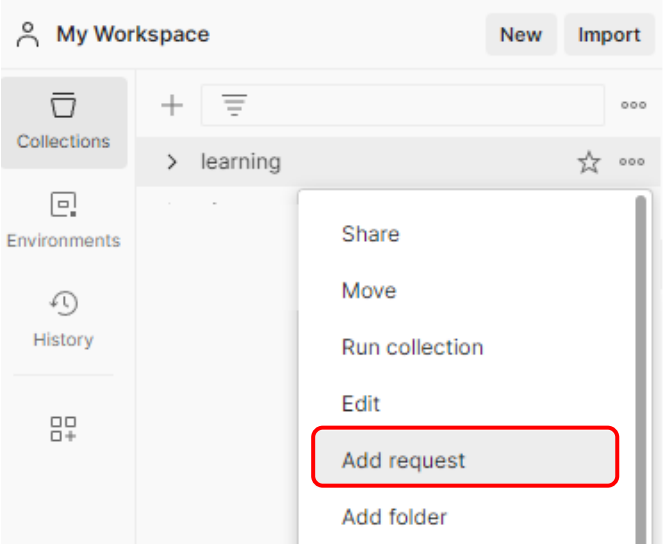


Remember to save your changes.

5. Under Collections, add a new collection **learning**.



6. Add request to the learning collection.

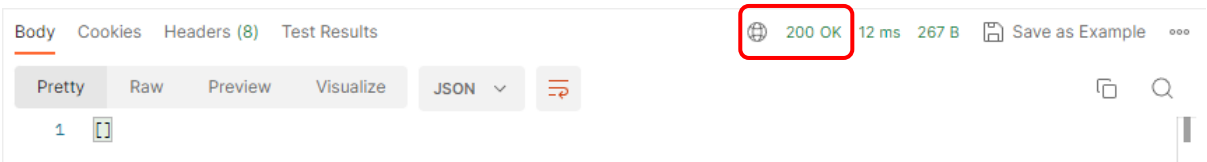


Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

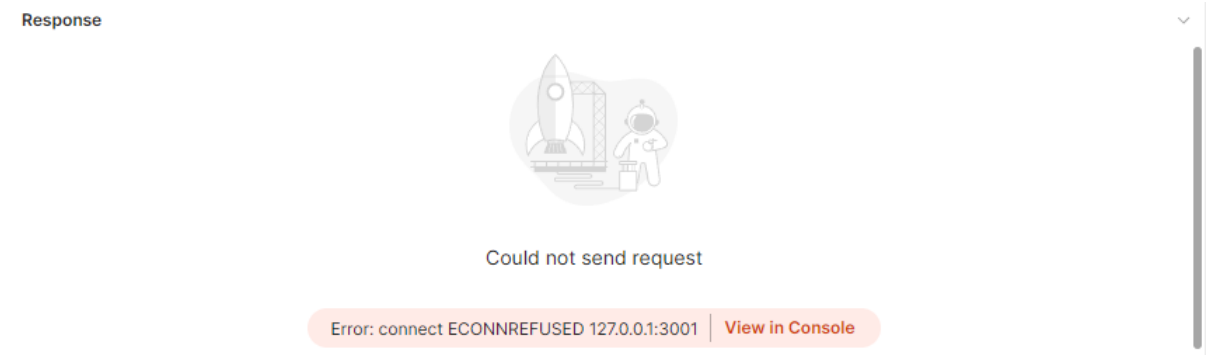
7. Prepare the Get request as below. You can save the request so you can test it later.

Method: GET (default)
URL: {{baseUrl}}/tutorial

Click button Send. Observe that the response status is 200 OK, the response body is an empty array, since no tutorial is added yet.



If your server is not running, you may see the error as below.

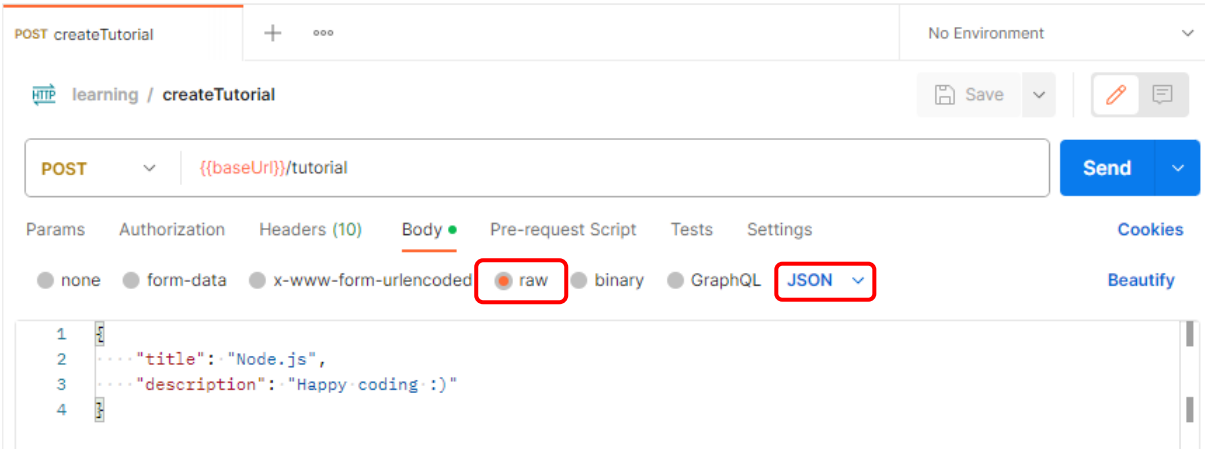


8. Add a new Post request as below.

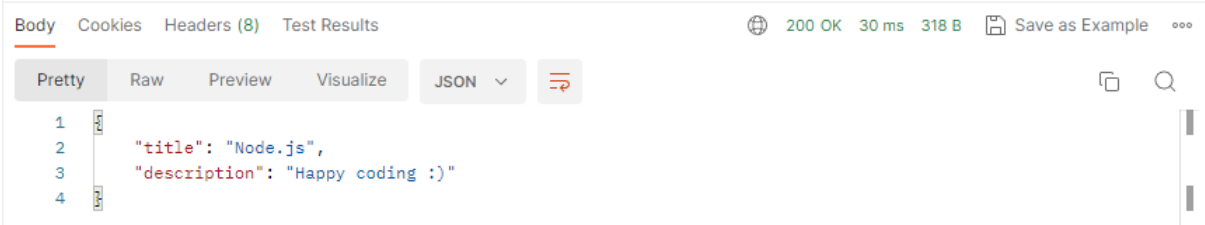
Method: POST
URL: {{baseUrl}}/tutorial
Body: (select **raw** and **JSON**)

```
{
    "title": "Node.js",
    "description": "Happy coding :)"
}
```

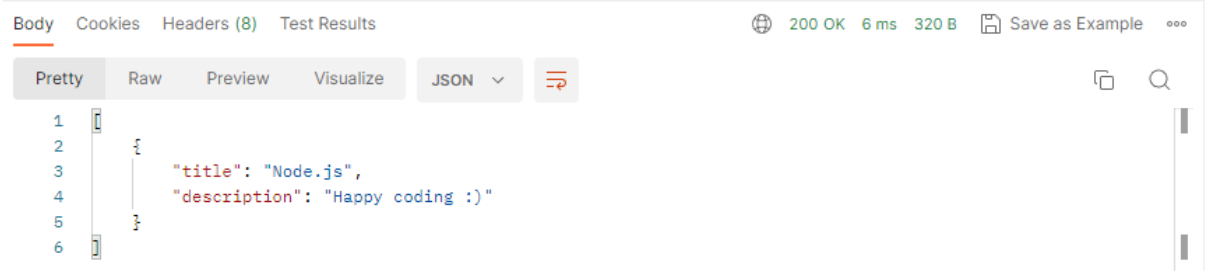

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.



Send request and observe the response body. The data is added to the list.



9. Test the Get request again. Now the response body returns the list with the added tutorial.



10. Restart the server and test the Get request. Notice that the list is reset to empty. This is because no persistent storage is used. We will learn how to use the database in the next practical.

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.